W5 COS30045

Lecture

Agenda

- 5.1 Lines and arcs
- 5.2 Pie and stack

Class

Agenda

- D3 Lines and Curves
- D3 Pie and Stack

W5.1 Drawing with lines and arcs

Lecture

Agenda

- 5.1 Lines
- 5.2 Pie and stack

Class

Agenda

- D3 Lines and Curves
- D3 Pie and Stack

Line and donut chart – but for 2024 Melbourne weather

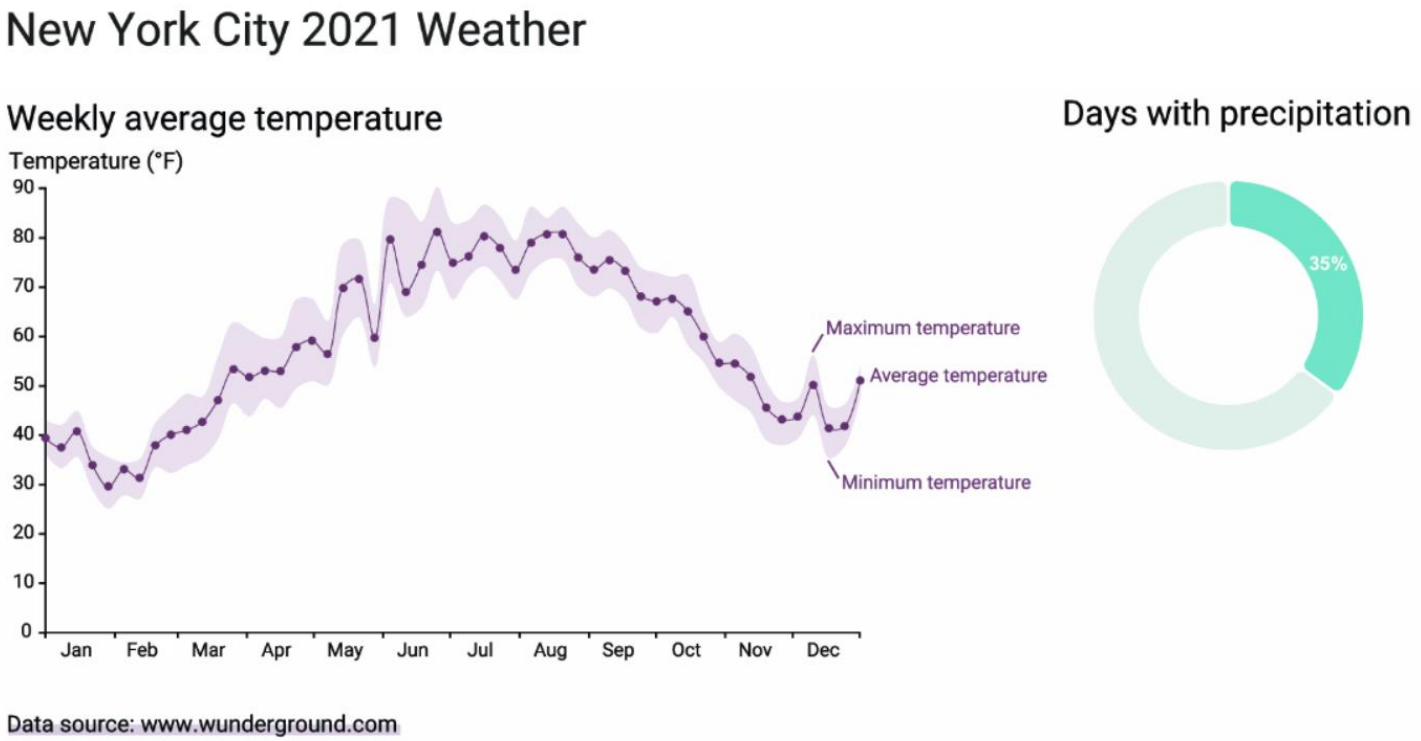


Figure 4.1 Project that we'll build in this chapter: a line chart of the temperature evolution in New York City in 2021 and a set of arcs showing the percentage of days with precipitation

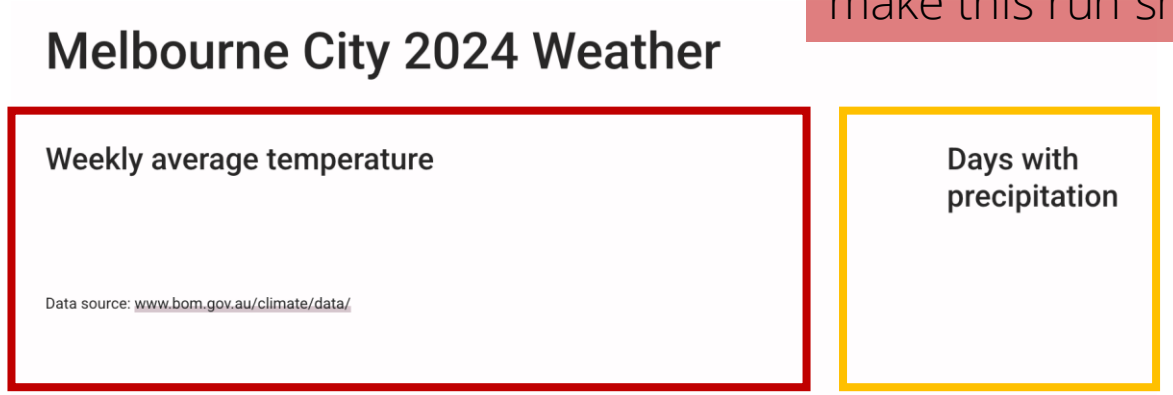
Laying out the charts using **col**

Fair amount of css code to make this run smoothly…

```
<body>
    <div class="container">

        <h1>Melbourne City 2024 Weather</h1>
        <div class="row">
            <div class="col col-9">
                <h2>Weekly average temperature</h2>
                <div class="responsive-svg-container">
                    <div id="line-chart"></div>
                </div>
            </div>
            <div class="col col-3">
                <h2>Days with precipitation</h2>
                <div class="responsive-svg-container small">
                    <div id="arc"></div>
                </div>
            </div>
        </div>


        <div class="source">Data source: <a
                href="http://www.bom.gov.au/climate/data/">www.bom.gov.au/climate/data/</a>
        </div>
    </div>
```



css > # visualization.css > .responsive-svg-container.small

```
/* Responsive container of the svg element */
.responsive-svg-container {
  width: 100%;
  max-width: 800px;
}
.responsive-svg-container.small {
  margin: 0 auto;
  max-width: 300px;
}
```

```
.container {
  margin-right: auto;
  margin-left: auto;
  padding-right: 30px;
  padding-left: 30px;
  width: 100%;
  max-width: 1200px;
}
.row {
  display: flex;
  flex-wrap: wrap;
  margin-left: -15px;
  margin-right: -15px;
}
.col {
  position: relative;
  width: 100%;
  min-height: 1px;
  padding-right: 15px;
  padding-left: 15px;
  flex-basis: 0;
  flex-grow: 1;
  max-width: 100%;
}
.col-9 {
  flex: 0 0 75%;
  max-width: 75%;
}
.col-3 {
  flex: 0 0 25%;
  max-width: 25%;
}
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Data set BOM temperature data Melb 2024

Rows: 62 | Columns: 14

#	RowID	Date *String*	Min-Max *String*	Jan *Number (Float)*	Feb *Number (Float)*	Mar *Number (Float)*	Apr *Number (Float*
1	Row0	1st	Max	23.2	24.3	28.1	28.2
2	Row1	2nd	Max	24.7	20.9	20.7	21.4
3	Row2	3rd	Max	26.9	33.4	20.9	16.6
4	Row3	4th	Max	21.9	37.5	20.7	18.9
5	Row4	5th	Max	28.9	19.9	32.1	20.6
6	Row5	6th	Max	32.6	21.1	25.1	22.1
7	Row6	7th	Max	27.2	21.9	21.9	20.3
8	Row7	8th	Max	20.3	22.4	26	20.6
9	Row8	9th	Max	24.2	21.2	37.6	16.6
10	Row9	10th	Max	27.4	26.7	36.9	17.8
11	Row10	11th	Max	23.1	33.2	35.1	17.7
12	Row11	12th	Max	33.4	35.8	23.1	17.7
13	Row12	13th	Max	21.7	36.7	19.9	19

#	RowID	Date *String*	max_temp *Number (double)*	min_temp *Number (double)*	ave_temp *Number (double)*
1	Row0	2024-01-01	26.367	16.283	21.325
2	Row1	2024-01-07	25.329	18.086	21.707
3	Row2	2024-01-14	25.371	16.543	20.957
4	Row3	2024-01-21	24.771	15.2	19.986
5	Row4	2024-01-28	25.286	15.6	20.443
6	Row5	2024-02-04	24.386	15.671	20.029
7	Row6	2024-02-11	27.557	15.643	21.6
8	Row7	2024-02-18	26.457	17.6	22.029
9	Row8	2024-02-25	26.929	15.243	21.086

Open for Innovation KNIME

Data processing:

Original data set
- Min and max for every day of year

Transformations, including
- **Unpivot** to put mths to new column
- **Date&Time to String** to convert dates into Weeks of year
- **GroupBy** – weeks of year
  - => average temp for each week
- **Maths** => average temp **estimated** by average of Min and Max

SWINBURNE UNIVERSITY OF TECHNOLOGY

Loading data D3.autotype suggested method

js > JS line-chart.js > drawLineChart > margin

```
1  d3.csv("../data/MelbTempData2024.csv", d => {
2      return {
3          date: d3.timeParse("%Y-%m-%d")(d.date),
4          max_temp: +d.max_temp,
5          min_temp: +d.min_temp,
6          ave_temp: +d.ave_temp
7      };
8  }).then(data => {
9      console.log("temperature data", data);
10     drawLineChart(data);
11 });
```

Make separate **js** file for line chart

```
temperature data ▼Array(53) [ {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, … ]
  ▼0: Object { date: "2024-01-01", Max_Temp: "26.366666666666700", Min_Temp: "16.283333333333300", … }
      Average_Temp: "21.325000000000000"
      Max_Temp: "26.366666666666700"
      Min_Temp: "16.283333333333300"
      date: "2024-01-01"
    ▶<prototype>: Object { … }
  ▶1: Object { date: "2024-01-07", Max_Temp: "25.328571428571400", Min_Temp: "18.085714285714300", … }
  ▶2: Object { date: "2024-01-14", Max_Temp: "25.37142857142860", Min_Temp: "16.54285714285710", … }
  ▶3: Object { date: "2024-01-21", Max_Temp: "24.77142857142860", Min_Temp: "15.2", … }
  ▶4: Object { date: "2024-01-28", Max_Temp: "25.28571428571430", Min_Temp: "15.600000000000000", … }
  ▶5: Object { date: "2024-02-04", Max_Temp: "24.385714285714300", Min_Temp: "15.671428571428600", … }
  ▶6: Object { date: "2024-02-11", Max_Temp: "27.557142857142900", Min_Temp: "15.64285714285710", … }
```

Dates can be tricky to read in

Date *String*
2024-01-01
2024-01-07
2024-01-14

```
temperature data ▼Array(53) [ {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, … ]
  ▼0: Object { Max_Temp: 26.3666666666667, Min_Temp: 16.2833333333333, Average_Temp: 21.325, … }
      Average_Temp: 21.325
      Max_Temp: 26.3666666666667
      Min_Temp: 16.2833333333333
    ▶date: Date Mon Jan 01 2024 00:00:00 GMT+1100 (Australian Eastern Daylight Time)
    ▶<prototype>: Object { … }
  ▶1: Object { Max_Temp: 25.3285714285714, Min_Temp: 18.0857142857143, Average_Temp: 21.7071428571429, … }
  ▶2: Object { Max_Temp: 25.3714285714286, Min_Temp: 16.5428571428571, Average_Temp: 20.9571428571429, … }
  ▶3: Object { Max_Temp: 24.7714285714286, Min_Temp: 15.2, Average_Temp: 19.9857142857143, … }
  ▶4: Object { Max_Temp: 25.2857142857143, Min_Temp: 15.6, Average_Temp: 20.4428571428571, … }
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Margin convention

Leaves room for axis, labels, legends etc

Chart will be drawn in inner chart area

At start – rough estimate of what required

May not be something GitHub Copilot will think about – ask it to do it specifically.

Will help with layout issues later



Figure 4.3 The D3 margin convention sets the value of the margins at the top, right, bottom, and left of a chart.

Margins

```
const drawLineChart = (data) => {
    const margin = {top: 40, right: 170, bottom: 25, left: 40};
    const width = 1000;
    const height = 500;
    const innerWidth = width - margin.left - margin.right;
    const innerHeight = height - margin.top - margin.bottom;
```

Using constants help if we need to modify later

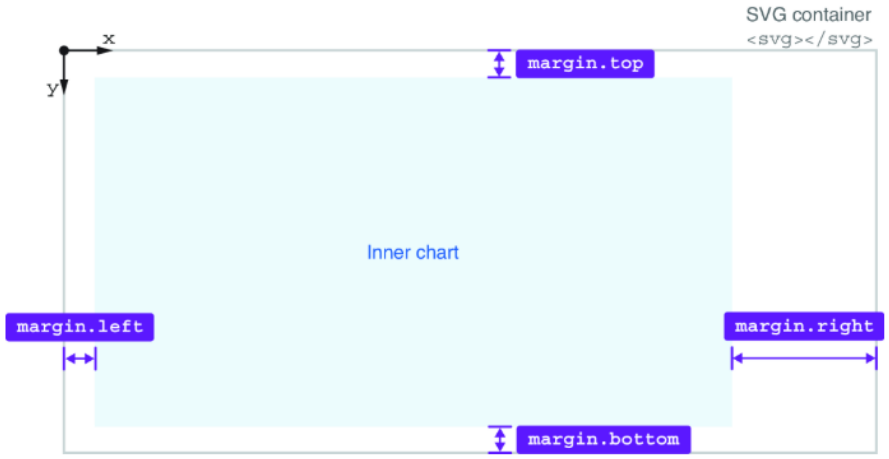


Figure 4.3 The D3 margin convention sets the value of the margins at the top, right, bottom, and left of a chart.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Create svg

```
const svg = d3.select("#line-chart")
 .append("svg")
 .attr("viewBox", `0, 0, ${width}, ${height}`);
```



Figure 4.5 Translation applied to the SVG group that will contain the inner chart, creating a new coordinate system for the elements contained in the inner chart

Make it so our chart is placed with origin at inner chart

```
const innerChart = svg
  .append("g")
  .attr("transform", `translate(${margin.left}, ${margin.top})`)
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Generating axes

Axes are generated by **axis()** using scales as input

Our chart will have:

- x-axis
  - Dates (time)
  - continuous scale
- y-axis
  - Temperature
  - continuous scale

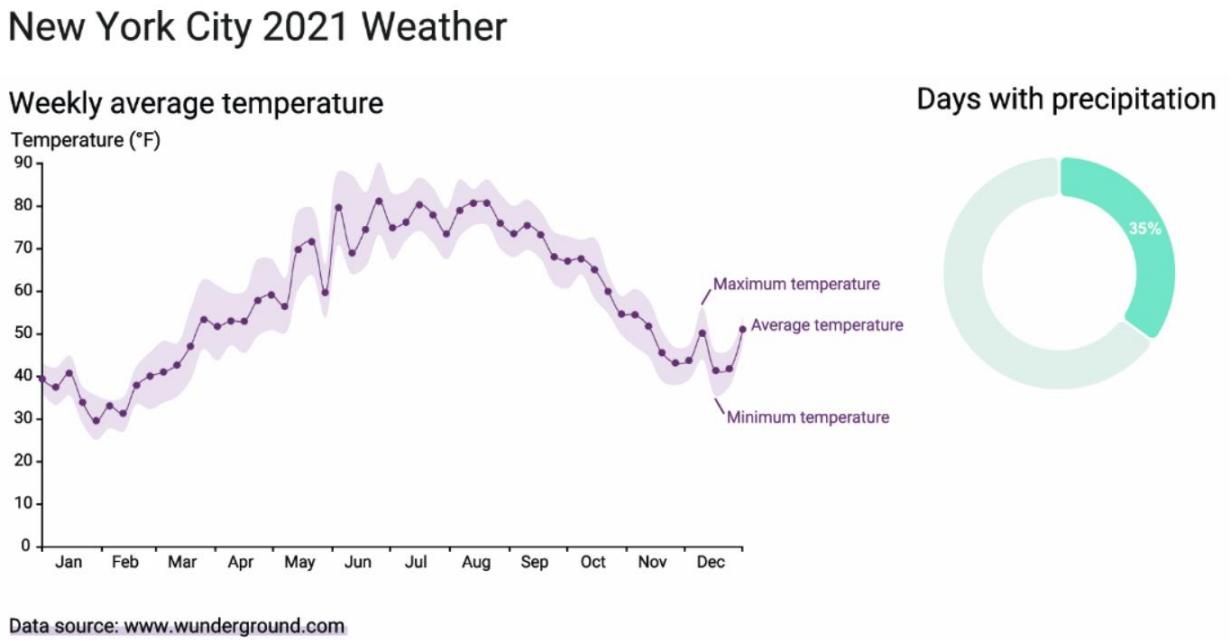


Figure 4.1 Project that we'll build in this chapter: a line chart of the temperature evolution in New York City in 2021 and a set of arcs showing the percentage of days with precipitation

SWINBURNE UNIVERSITY OF TECHNOLOGY

Generating axes

x-Axis

- Need to know first date and last date to determine length of domain
- Scale function then fits this to **innerWidth**

```
const firstDate = new Date(2024, 0, 1);
const lastDate = d3.max(data, d => d.date);
const xScale = d3.scaleTime()
  .domain([firstDate, lastDate])
  .range([0, innerWidth]);
```

JavaScript code

```
new Date(year, month, day)
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Generating axes

y-Axis

- Need to know max ave_temp determine length of domain
- Min is assumed to be 0
- Scale function then fits this to **innerHeight**

```
const maxTemp = d3.max(data, d => d.ave_temp);
const yScale = d3.scaleLinear()
  .domain([0, maxTemp])
  .range([innerHeight, 0]);
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Appending axes

D3 Axis generators:

- axisTop(),
- axisRight(),
- axisBottom(),
- axisLeft()

Take scales as inputs




```
const bottomAxis = d3.axisBottom(xScale);
```

Call to screen:

```
innerChart
 .append("g")
 .attr("class", "axis-x")
 .call(bottomAxis);
```

Append the scale to innerChart

SWINBURNE UNIVERSITY OF TECHNOLOGY

Appending axes

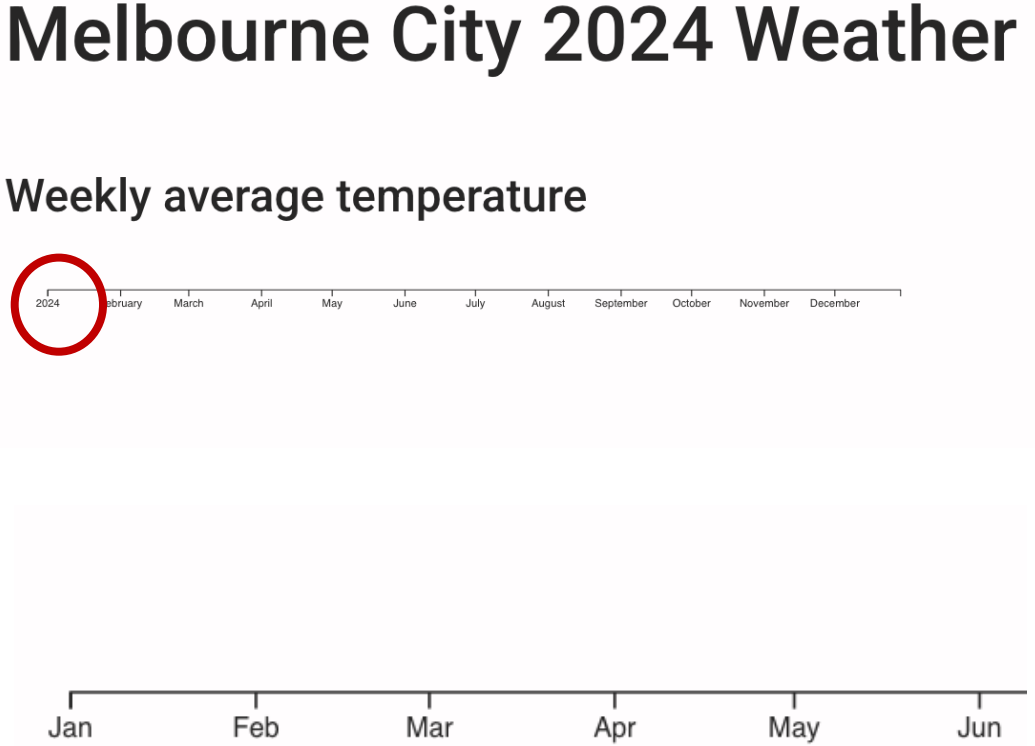
GitHub Copilot

Why is first month in my axis 2024 instead of January?

=> Default behaviour

How do I fix this?

=> Format ticks explicitly




```
const bottomAxis = d3.axisBottom(xScale)
 .tickFormat(d3.timeFormat("%b"));
```

%b – use 3 letter abbreviation
%m – use month letter
%Y – use full year
%y – use two digit year….

SWINBURNE UNIVERSITY OF TECHNOLOGY

Appending axes – x-axis

Default axis at origin

Need to translate to put it at bottom

See Dufour & Meeks (2024) Ch 4
for more ways to customise axis label
and tick position

```
innerChart
  .append("g")
  .attr("class", "axis-x")
  .attr("transform", `translate(0, ${innerHeight})`)
  .call(bottomAxis);
```

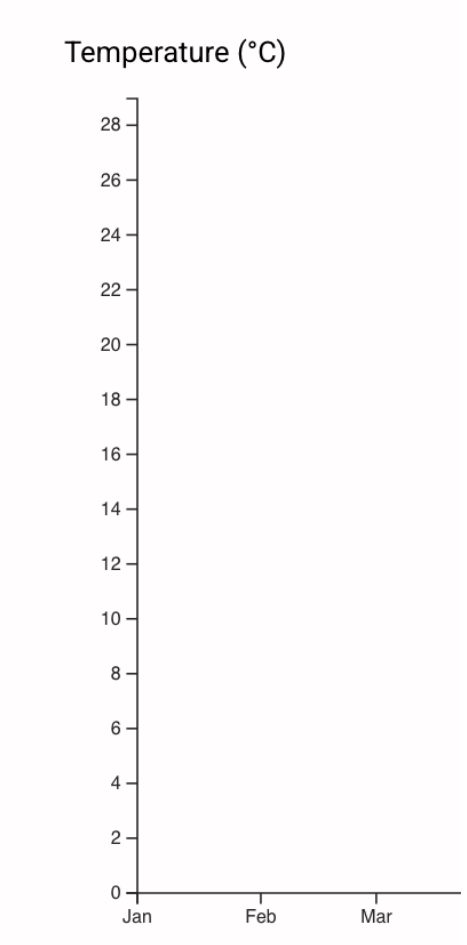


Figure 4.6 By default, D3 axes are generated at the origin of the selection, which is the top-left corner of the inner chart in this example. We need to apply a translation to move them to the desired location.

Appending axes y-axis

```
const leftAxis = d3.axisLeft(yScale);

    innerChart
      .append("g")
      .attr("class", "axis-y")
      .call(leftAxis);
```



Adding axis labels

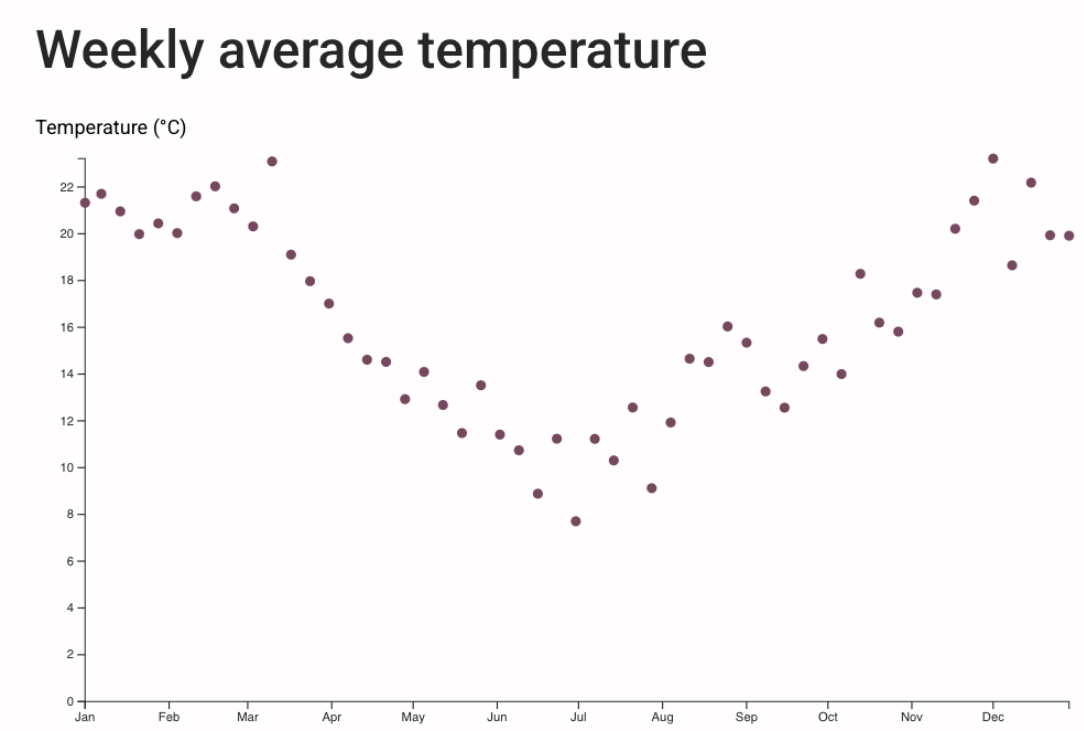
```
    svg
      .append("text")
      .text("Temperature (°C)")
      .attr("y", 20);
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

Draw a scatter plot

```
const aubergine = "#75485E";

innerChart
 .selectAll("circle")
 .data(data)
 .join("circle")
  .attr("r", 4)
  .attr("cx", d => xScale(d.date))
  .attr("cy", d => yScale(d.ave_temp))
  .attr("fill", aubergine);
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Drawing a line

```
d3.line()
  .x( d => xScale(d.xValue) )
  .y( d => yScale(d.yValue) );
```

Horizontal position of each data point

Vertical position of each data point

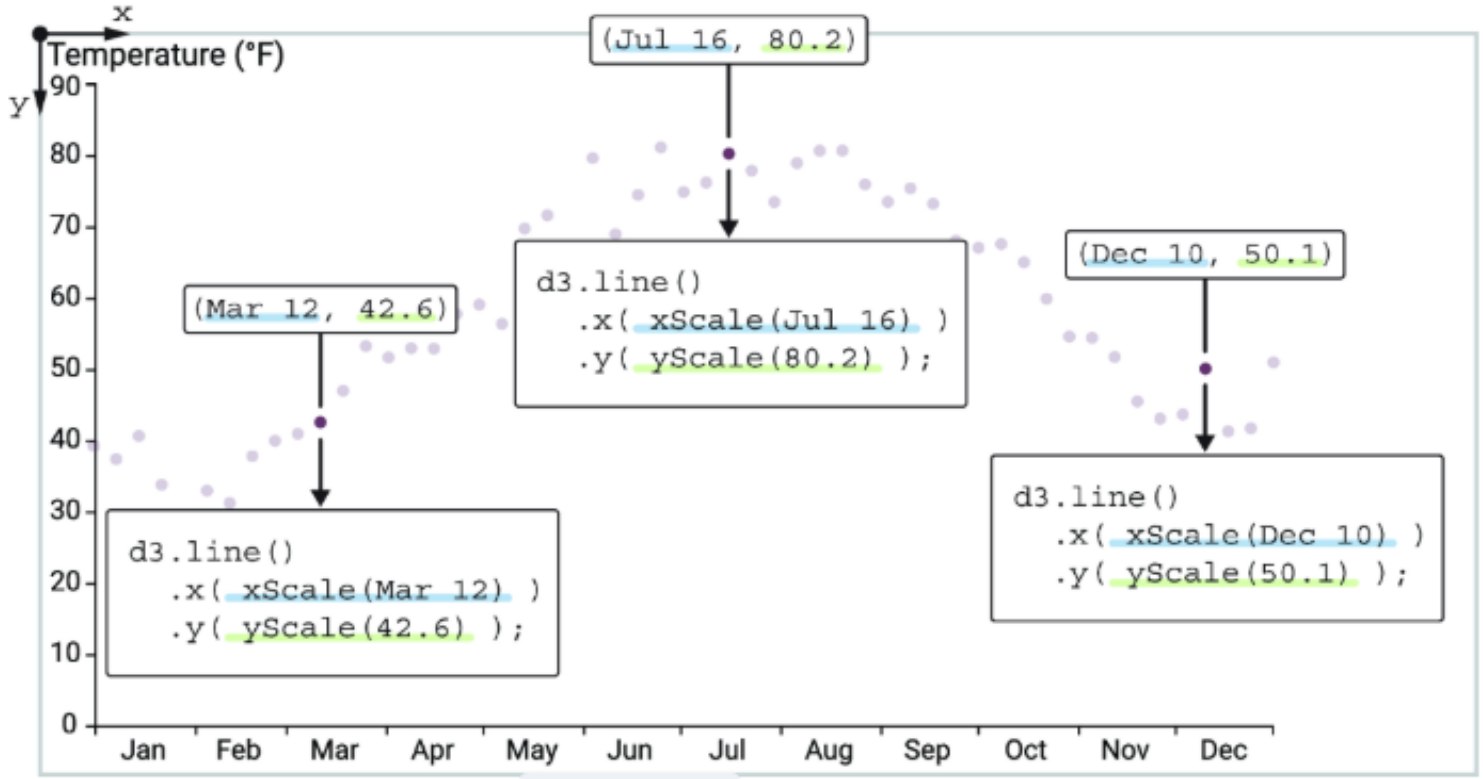


Figure 4.14 Line generator `d3.line()` is used in combination with two accessor functions, `x()` and `y()`, that respectively take the horizontal and vertical position of each data point as arguments.

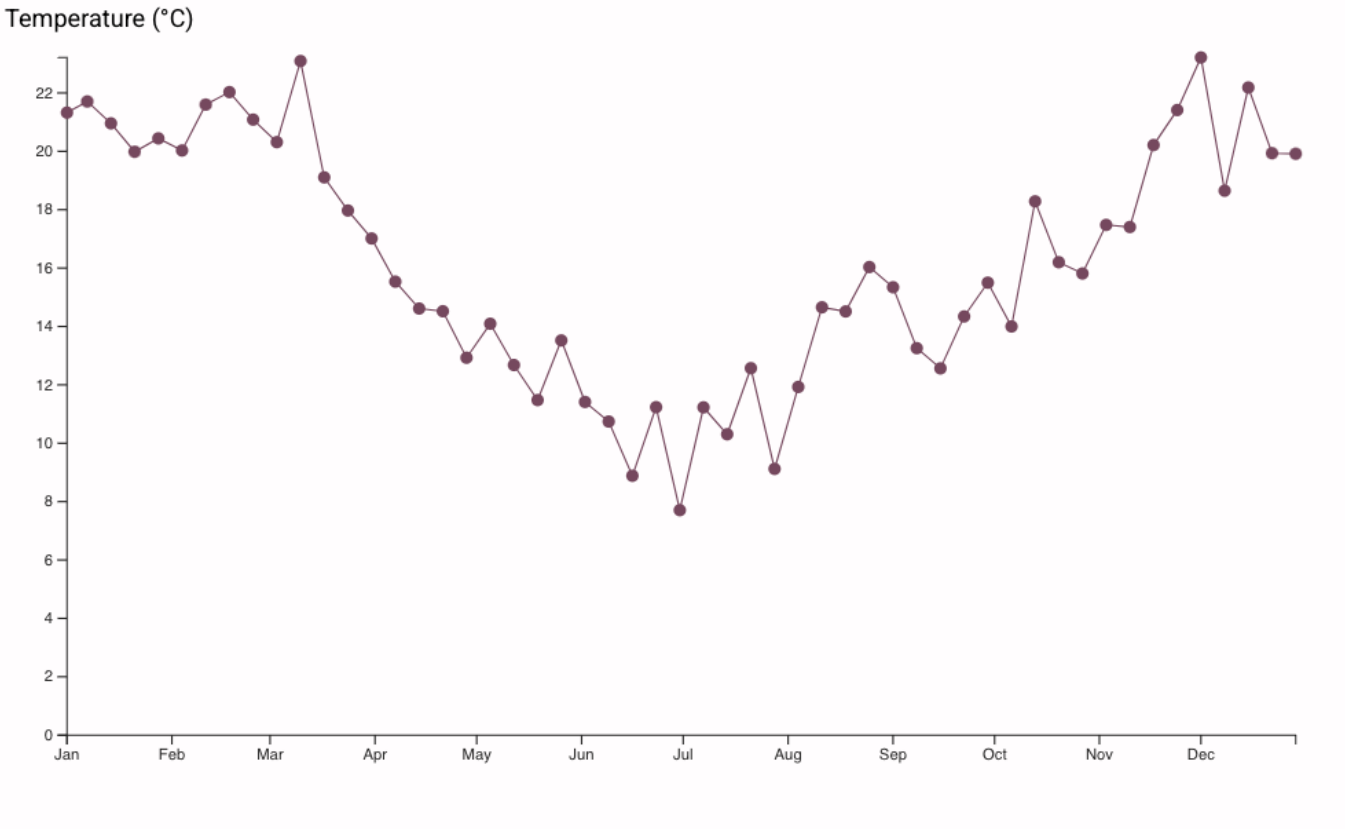
SWINBURNE UNIVERSITY OF TECHNOLOGY

Draw the line

```
const lineGenerator = d3.line()
  .x(d => xScale(d.date))
  .y(d => yScale(d.ave_temp));
```

Default line generator is:
`.curve(d3.curveLinear);`

```
innerChart
  .append("path")
  .attr("d", lineGenerator(data))
  .attr("fill", "none")
  .attr("stroke", aubergine);
```

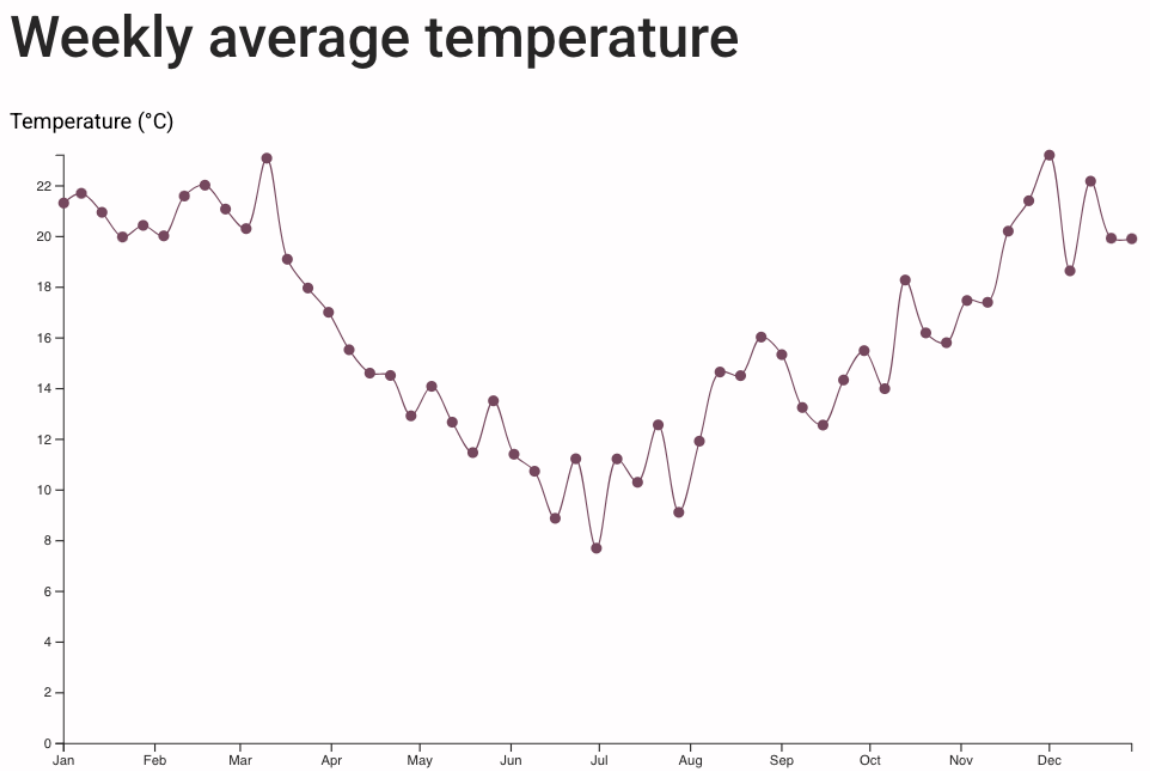


SWINBURNE UNIVERSITY OF TECHNOLOGY

Interpolating data points to curve

```
const lineGenerator = d3.line()
  .x(d => xScale(d.date))
  .y(d => yScale(d.ave_temp))
  .curve(d3.curveCatmullRom);


innerChart
  .append("path")
  .attr("d", lineGenerator(data))
  .attr("fill", "none")
  .attr("stroke", aubergine)
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Interpolating data points to curve

d3.curveBasics()

- Underestimates sudden variations

d3.curve Bundle()

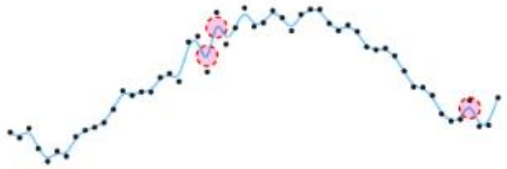
- straightens curve

d3.curveMonotoneX, d3.curveCatmullRom

- Follows data points closely

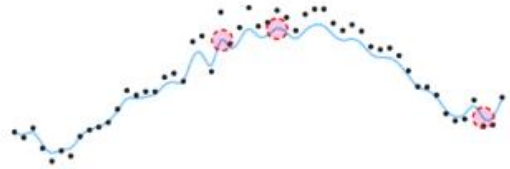
d3.curveStep

Draw points first, then check curve represents them accurately



`d3.curveBasis`

Produces a cubic basis spline that passes through the first and the last data point



`d3.curveBundle`

Produces a straightened cubic basis spline



`d3.curveCardinal`

Produces a cardinal spline that passes through each data point



`d3.curveCatmullRom`

Produces a cubic Catmull–Rom spline that passes through each data point



`d3.curveMonotoneX`

Produces a cubic spline that passes through each data point while preserving monotonicity in the vertical direction



`d3.curveStep`

Produces a step function passing through each data point and alternating horizontal and vertical lines

Figure 4.17 Different curve interpolations and how they modify the representation of data

Dufour & Meeks (2024) Ch 4

SWINBURNE UNIVERSITY OF TECHNOLOGY

Overview of drawing line chart

```
const lineGenerator = d3.line()
 .x(d => xScale(d.date))
 .y(d => yScale(d.ave_temp));
```

```
const lineGenerator = d3.line()
 .x(d => xScale(d.date))
 .y(d => yScale(d.ave_temp))
 .curve(d3.curveCatmullRom);
```

```
innerChart
 .append("path")
  .attr("d", lineGenerator(data))
  .attr("fill", "none")
 . attr("stroke", aubergine);
```

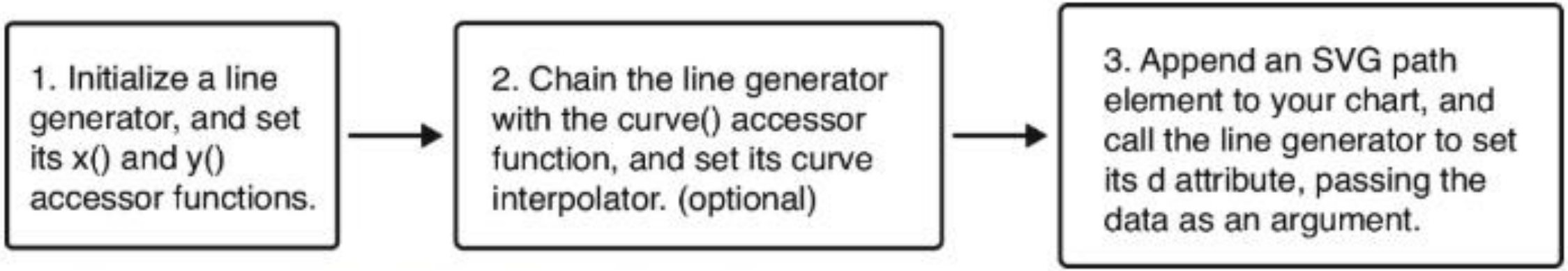


Figure 4.18 Steps to create a line chart

SWINBURNE UNIVERSITY OF TECHNOLOGY

Drawing an area

Set up area generator

```
const areaGenerator = d3.area()
  .x(d => xScale(d.date))
  .y0(d => yScale(d.min_temp))
  .y1(d => yScale(d.max_temp));
```

x – calculates horizontal position of data point

y0 - lower edge of area

y1- upper edge of area

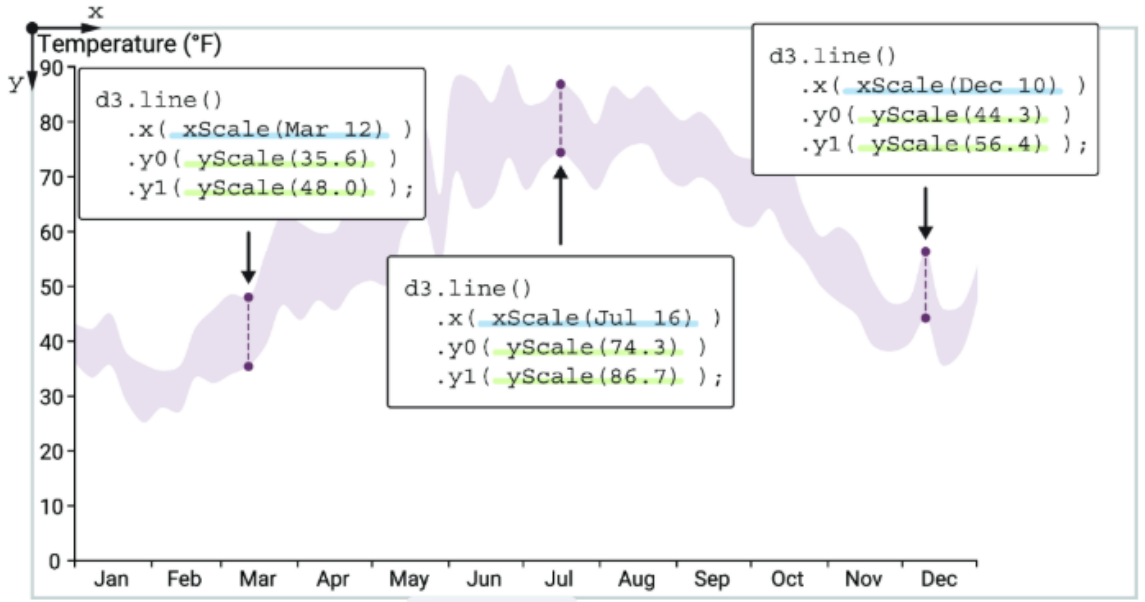
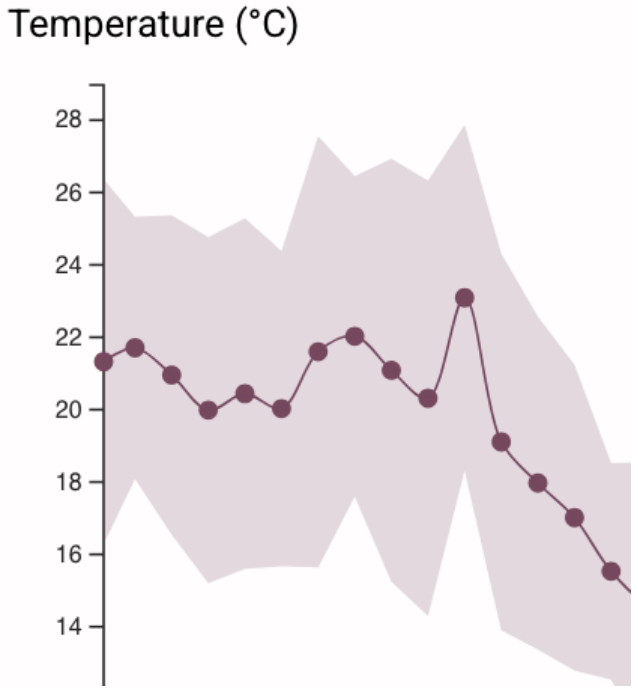


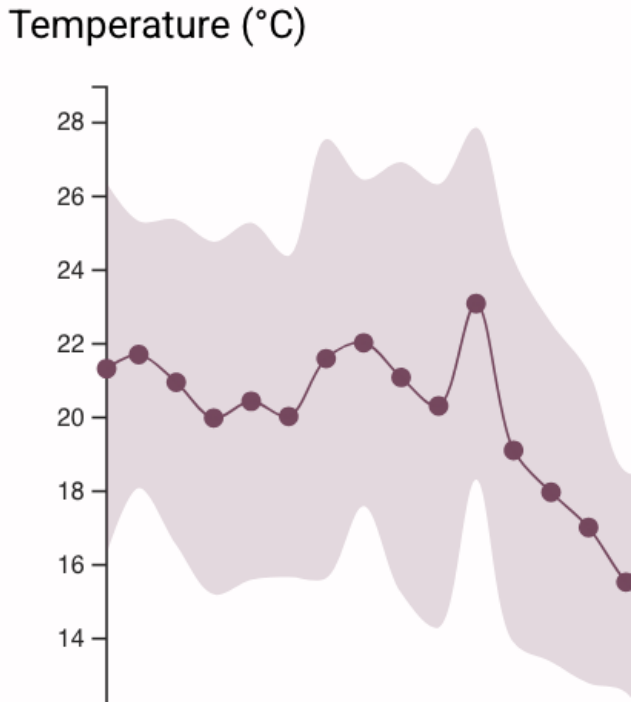
Figure 4.19 Area generator `d3.area()` is used in combination with three or more accessor functions. To draw the area between the minimum and the maximum temperatures, we use `x()`, `y0()`, and `y1()`. The first one calculates the horizontal position of each data point, the second the vertical position of the data points on the lower boundary (here, the minimum temperature), and the third one the vertical position of the data points on the upper edge (here, the maximum temperature).

Drawing area chart

Append path determined by areaGenerator

```
innerChart
  .append("path")
  .attr("d", areaGenerator(data))
  .attr("fill", aubergine)
  .attr("fill-opacity", 0.2);
```





Add to areaGenerator

```
.curve(d3.curveCatmullRom);
```

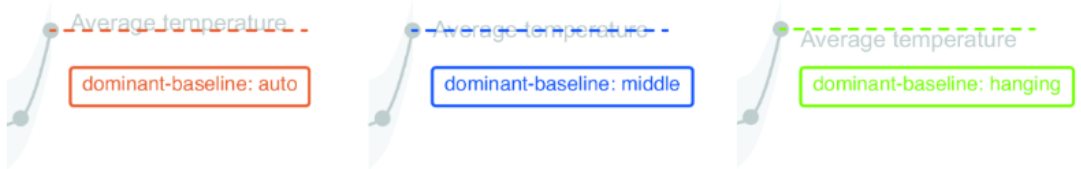

SWINBURNE UNIVERSITY OF TECHNOLOGY

Adding Labels - Average

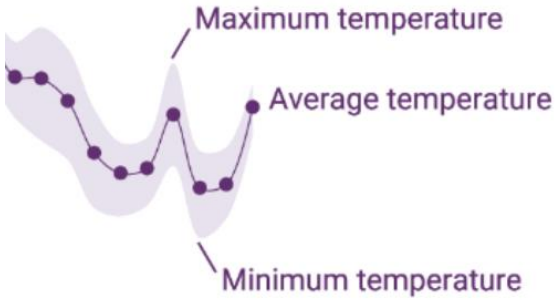
x - End of line -> lastDate (plus some padding)

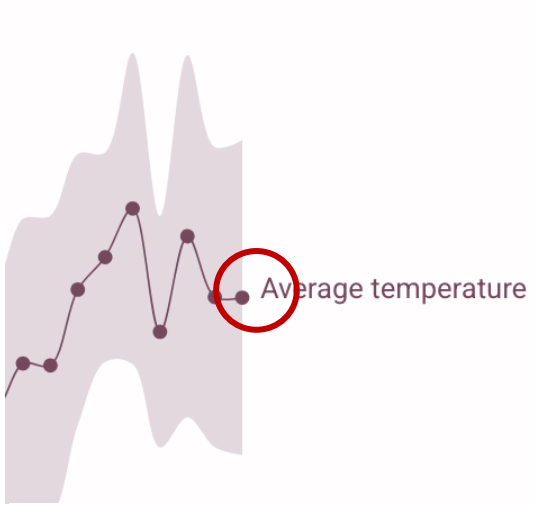
y - Find last row and access average temp

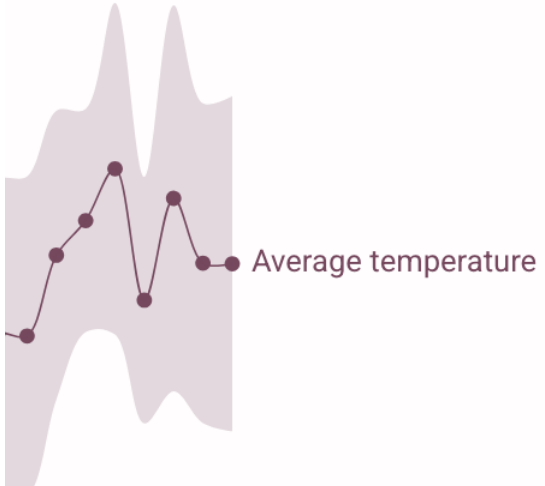
```
innerChart
  .append("text")
  .text("Average temperature")
  .attr("x", xScale(lastDate) + 10)
  .attr("y", yScale(data[data.length - 1].ave_temp))
  .attr("fill", aubergine);
```




```
.attr("dominant-baseline", "middle")
```





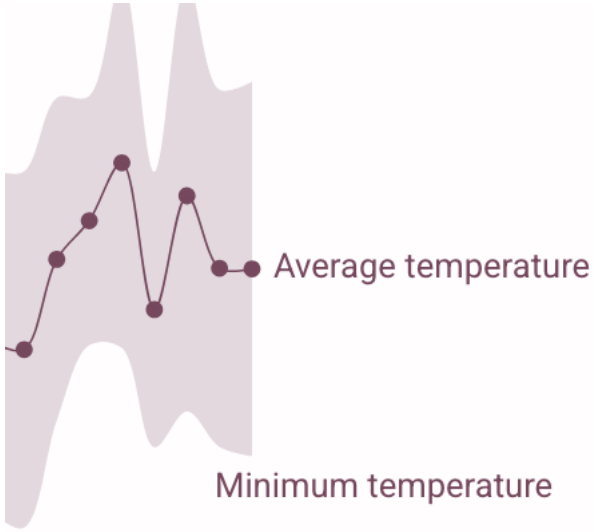


Adding Labels – Minimum

Add label for minimum temperature

Offset from Average temperature label

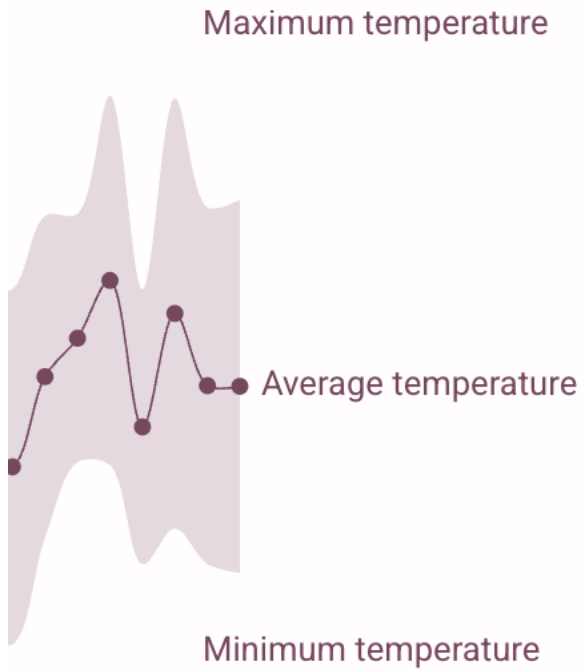
```
innerChart
 .append("text")
 .text("Minimum temperature")
 .attr("x", xScale(data[data.length - 3].date) + 13)
 .attr("y", yScale(data[data.length - 3].min_temp) + 40)
 .attr("alignment-baseline", "hanging")
 .attr("fill", aubergine);
```



- Label x and y position is 3 data points in form end of data set
- Use + to get in desired position

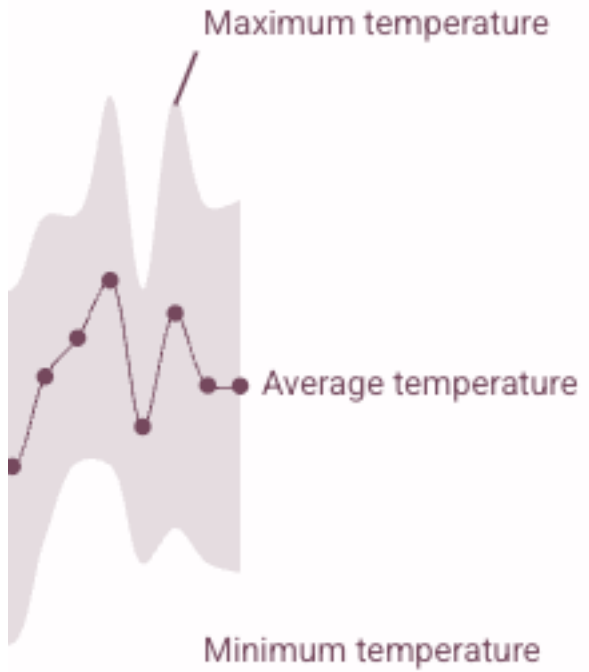
Adding labels – Maximum

```
innerChart
  .append("text")
  .text("Maximum temperature")
  .attr("x", xScale(data[data.length - 3].date) + 13)
  .attr("y", yScale(data[data.length - 3].max_temp) + -30)
  .attr("fill", aubergine);
```

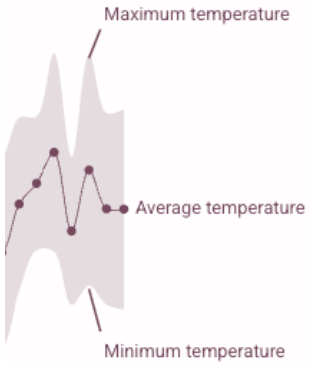


Adding labels – Lines

```
// max label line
innerChart
  .append("line")
  .attr("x1", xScale(data[data.length - 3].date))
  .attr("y1", yScale(data[data.length - 3].max_temp) + 3)
  .attr("x2", xScale(data[data.length - 3].date) + 10)
  .attr("y2", yScale(data[data.length - 3].max_temp) + -22)
  .attr("stroke", aubergine)
  .attr("stroke-width", 2);
```




```
// min label line
innerChart
.append("line")
  .attr("x1", xScale(data[data.length - 3].date))
  .attr("y1", yScale(data[data.length - 3].min_temp) + 3)
  .attr("x2", xScale(data[data.length - 3].date) + 10)
  .attr("y2", yScale(data[data.length - 3].min_temp) + 40)
  .attr("stroke", aubergine)
  .attr("stroke-width", 2);
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

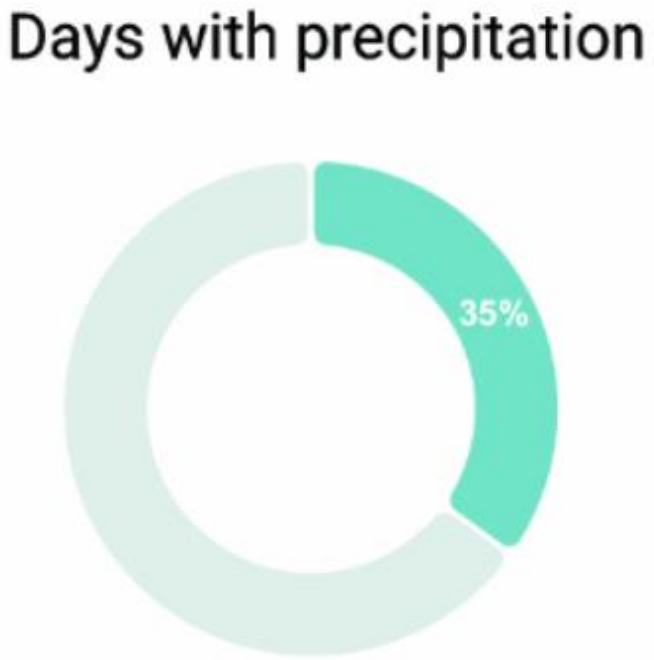
Drawing Arcs

Create new js file for arcs

Load data set

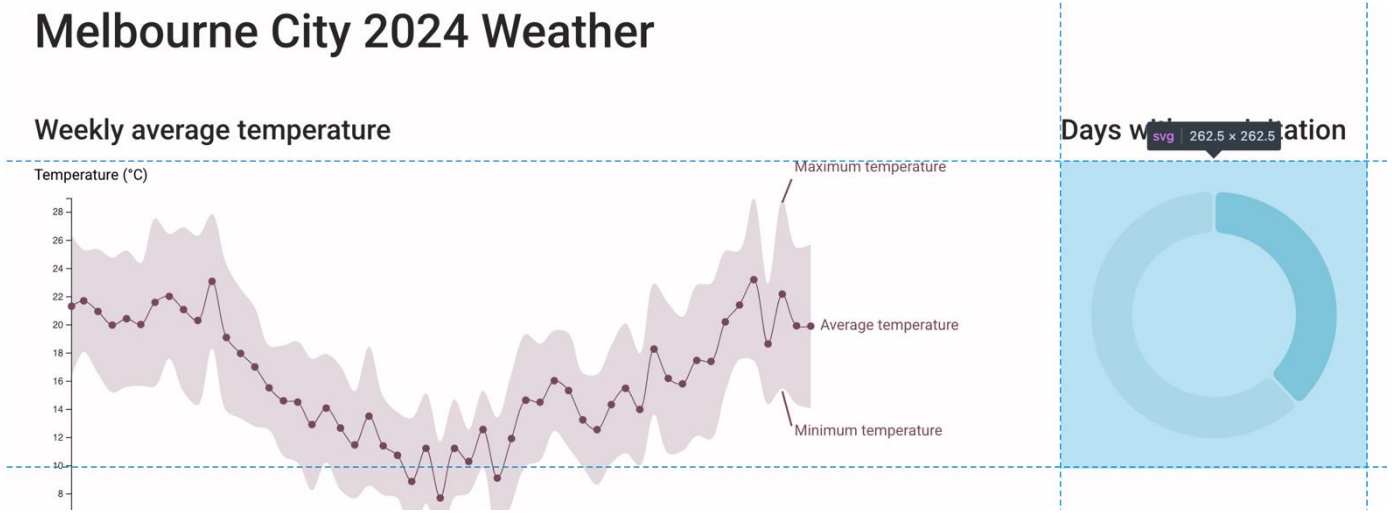
```
d3.csv("../data/2024MelbRainfall.csv", d3.autoType).then(data => {
  console.log("precipitation data", data);
  drawArc(data);
});
```

```
precipitation data ▼Array(365) [ {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, … ]                arcs
                   ▼[0…99]
                     ▶0: Object { date: Date Mon Jan 01 2024 11:00:00 GMT+1100 (Australian Eastern Daylight Time), rainfall: 0 }
                     ▶1: Object { date: Date Thu Feb 01 2024 11:00:00 GMT+1100 (Australian Eastern Daylight Time), rainfall: 0 }
                     ▶2: Object { date: Date Fri Mar 01 2024 11:00:00 GMT+1100 (Australian Eastern Daylight Time), rainfall: 0 }
                     ▶3: Object { date: Date Mon Apr 01 2024 11:00:00 GMT+1100 (Australian Eastern Daylight Time), rainfall: 0 }
                     ▶4: Object { date: Date Wed May 01 2024 10:00:00 GMT+1000 (Australian Eastern Standard Time), rainfall: 0.2 }
                     ▶5: Object { date: Date Sat Jun 01 2024 10:00:00 GMT+1000 (Australian Eastern Standard Time), rainfall: 0 }
                     ▶6: Object { date: Date Mon Jul 01 2024 10:00:00 GMT+1000 (Australian Eastern Standard Time), rainfall: 4.4 }
                     ▶7: Object { date: Date Thu Aug 01 2024 10:00:00 GMT+1000 (Australian Eastern Standard Time), rainfall: 0.2 }
                     ▶8: Object { date: Date Sun Sep 01 2024 10:00:00 GMT+1000 (Australian Eastern Standard Time), rainfall: 0.2 }
```



SWINBURNE UNIVERSITY OF TECHNOLOGY

Create svg container for chart




```
<div class="col col-3">
<h2>Days with precipitation</h2>
<div class="responsive-svg-container small">
<div id="arc"></div>
</div>
```

index.html

```
const drawArc = (data) => {

  const pieChartWidth = 300;
  const pieChartHeight = 300;
  const svg = d3.select("#arc")
    .append("svg")
    .attr("viewBox", [0, 0, pieChartWidth, pieChartHeight]);

};
```

Dufour & Meeks (2024) Ch 4

SWINBURNE UNIVERSITY OF TECHNOLOGY

Polar coordinate system

No need to reserve space for axis or labels

Cartesian

- x
- y

Polar coordinate:

- Radius
- Angle

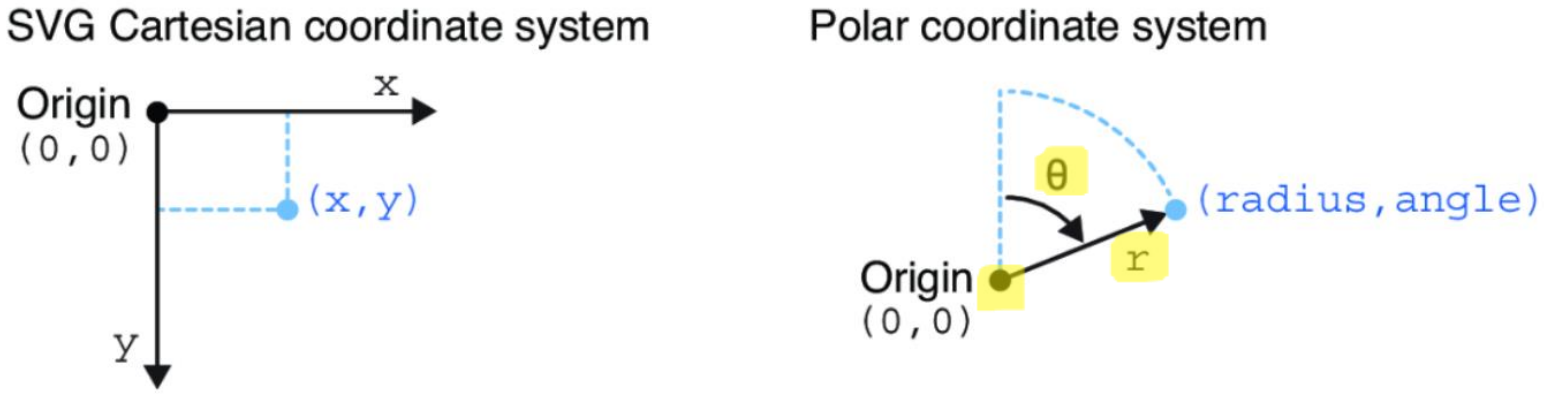


Figure 4.24 The dimensions of a Cartesian coordinate system are perpendicular to one another, while the polar coordinate system uses the radius and angle dimensions to describe a position in space.

Polar coordinate system

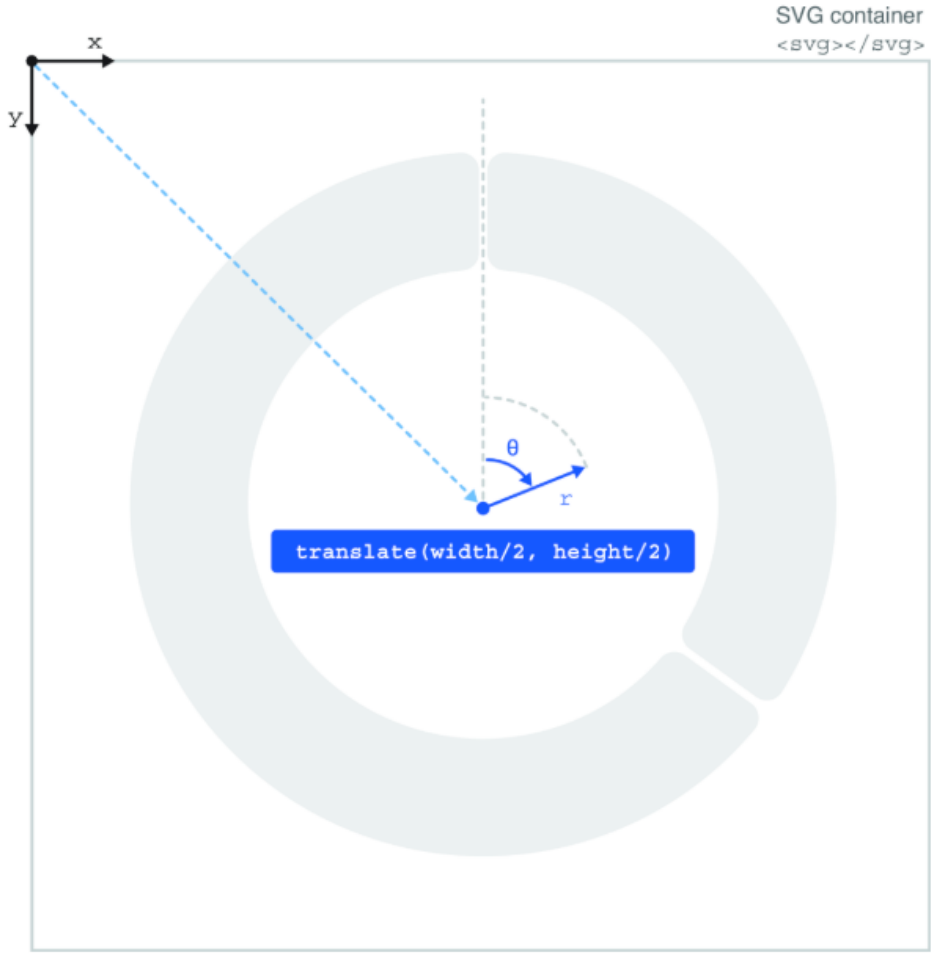


Figure 4.25 We facilitate the creation of a set of arcs by wrapping them into an SVG group and translating this group to the center of the SVG container. When we append arcs to the group, their position will be automatically relative to the center of the chart, which corresponds to the origin of their polar coordinate system.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Polar coordinate system

Make a group with origin in the middle of the svg

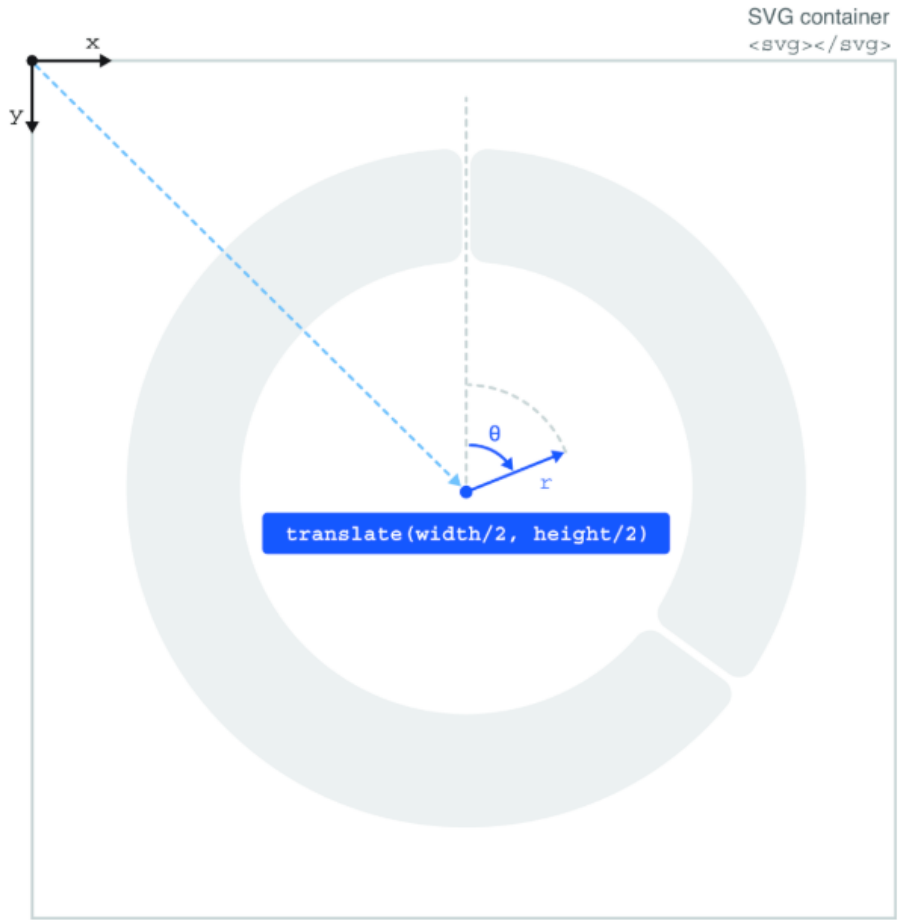


Figure 4.25 We facilitate the creation of a set of arcs by wrapping them into an SVG group and translating this group to the center of the SVG container. When we append arcs to the group, their position will be automatically relative to the center of the chart, which corresponds to the origin of their polar coordinate system.

```
const innerChart = svg
  .append("g")
  .attr("transform", `translate(${pieChartWidth/2}, ${pieChartHeight/2})`);
```

Polar coordinate system

Calculate the angle for days with rain

- Get length (365 days)
- Find length of set of data when rainfall is greater than 0
- Get percent of days with rain

Use pie layout generator for more than two categories

```
numberOfDays 365
numberOfDaysWithPrecipitation 138
percentageDaysWithPrecipitation 38
```

```
const numberOfDays = data.length;
const numberOfDaysWithPrecipitation = data.filter(d => d.rainfall > 0).length;
const percentageDaysWithPrecipitation = Math.round(numberOfDaysWithPrecipitation / numberOfDays * 100);
```

Find angle

360 = degrees in circle (css)

```
const angleDaysWithPrecipitation_deg = percentageDaysWithPrecipitation * 360 / 100;
const angleDaysWithPrecipitation_rad = angleDaysWithPrecipitation_deg * Math.PI / 180;
```

```
angleDaysWithPrecipitation_deg 136.8
angleDaysWithPrecipitation_rad 2.387610416728243
```

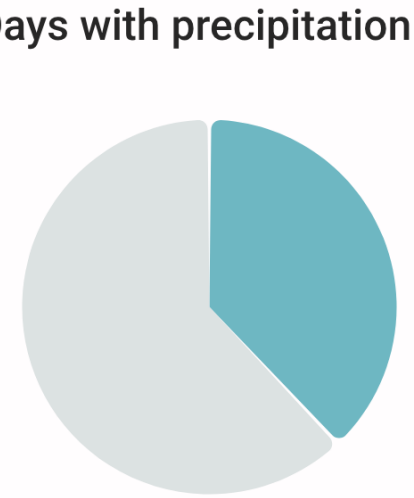
Convert to radians (JavaScript)

SWINBURNE UNIVERSITY OF TECHNOLOGY

Arc generator

To draw arc require:

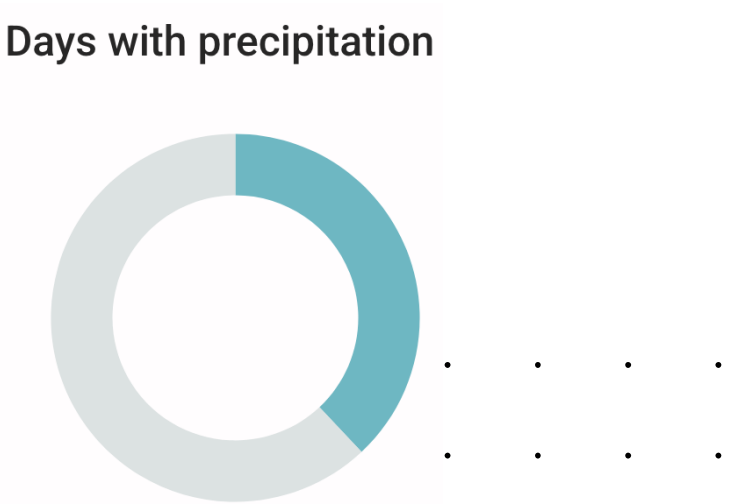
- Inner radius (when inner radius = 0 -> pie chart!
- Outer radius



For illustration only - arcs won't show until append a path

```
const arcGenerator = d3.arc()
  .innerRadius(80)
  .outerRadius(120)

  .padAngle(0.02)
  .cornerRadius(6);
```





SWINBURNE UNIVERSITY OF TECHNOLOGY

Arc generator

```
const arcGenerator = d3.arc()
  .innerRadius(80)
  .outerRadius(120)
  .padAngle(0.02)
  .cornerRadius(6);
```

```
d3.arc()
  .innerRadius( 80 )
  .outerRadius( 120 )
  .padAngle( 0.02 )
  .cornerRadius( 6 );
```

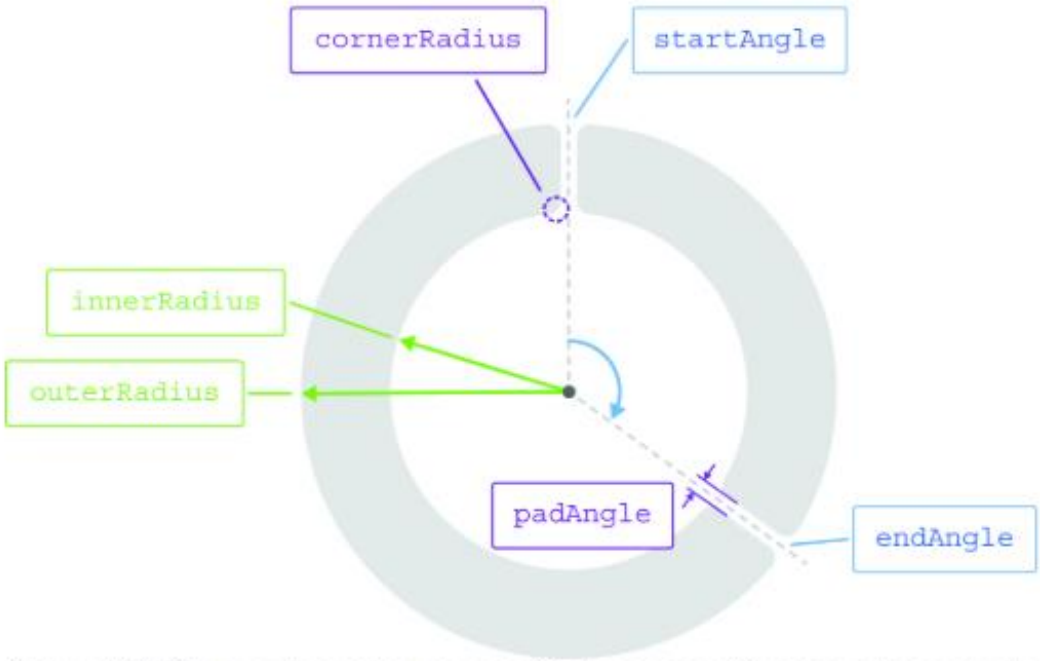


Figure 4.26 The arc generator uses multiple accessor functions to compute the `d` attribute of an arc. Here we set its inner radius, outer radius, padding angle, and corner radius during the generator declaration. We'll pass each arc's start and end angles when we append the path element to the chart.

SWINBURNE UNIVERSITY OF TECHNOLOGY

Append path with arcGenerator

Draw each arc separately



```
innerChart
 .append("path")
  .attr("d", () => {
  return arcGenerator({
    startAngle: 0,
    endAngle: angleDaysWithPrecipitation_rad
  });
 })
 .attr("fill", "#6EB7C2");
```

Start of circle - 12 o'clock

```
innerChart
 .append("path")
  .attr("d", () => {
  return arcGenerator({
    startAngle: angleDaysWithPrecipitation_rad,
    endAngle: 2 * Math.PI
  });
 })
 .attr("fill", "#DCE2E2");
```

End of circle

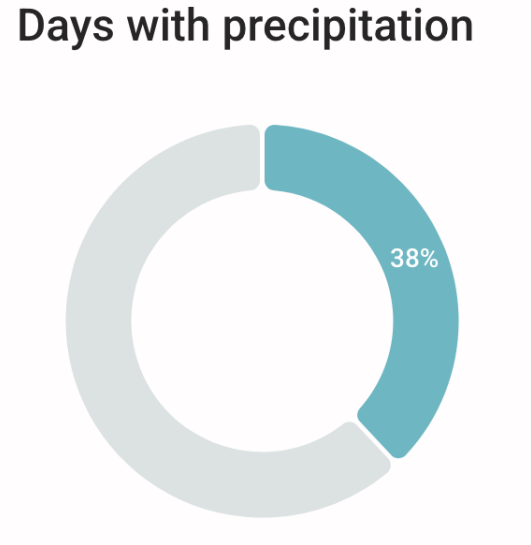
SWINBURNE UNIVERSITY OF TECHNOLOGY

Adding labels to arc at centroid ('centre of mass')

Calculate the centroid

```
const centroid = arcGenerator
  .startAngle(0)
  .endAngle(angleDaysWithPrecipitation_rad)
  .centroid();
```

```
centroid ▶ Array [ 92.97764858882515, -36.81245526846778 ]
```



Create the label

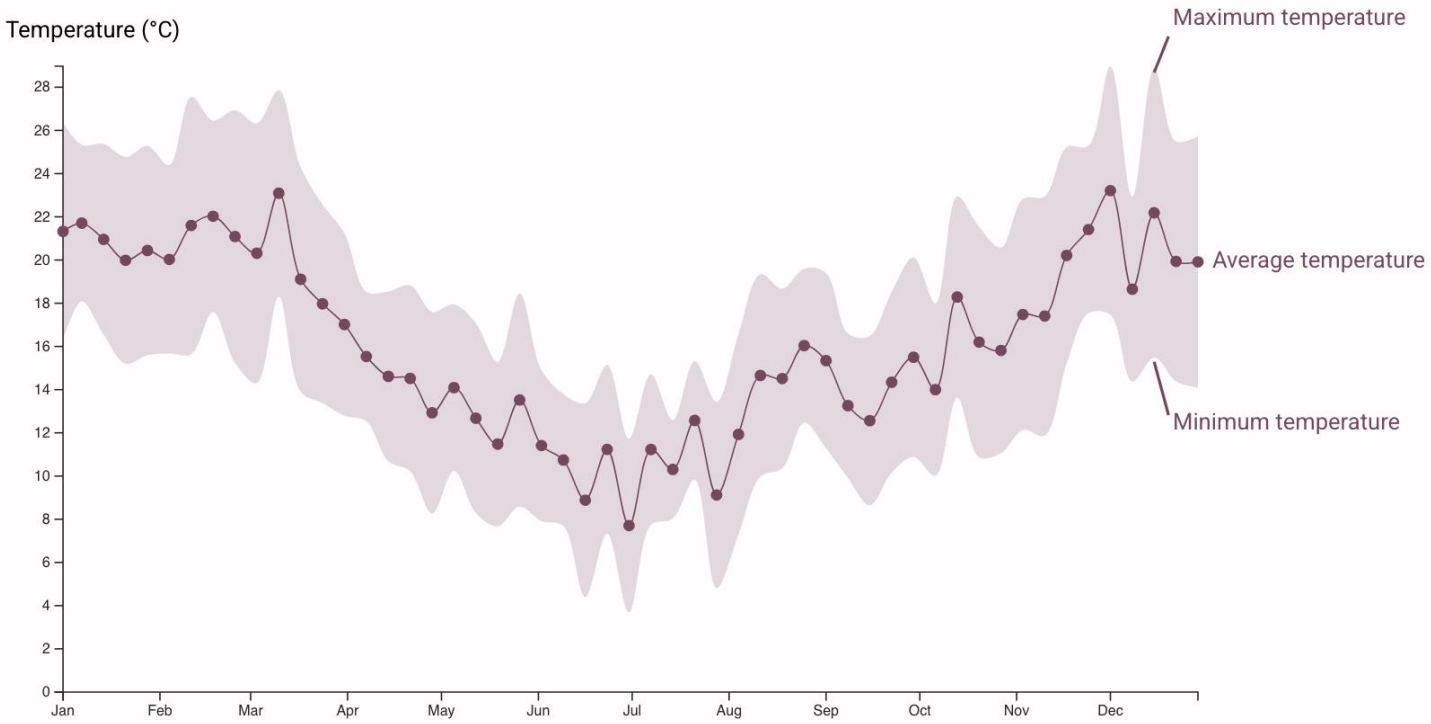
```
innerChart
 .append("text")
  .text(d => d3.format(".0%")(percentageDaysWithPrecipitation/100))
  .attr("x", centroid[0])
  .attr("y", centroid[1])
  .attr("text-anchor", "middle")
  .attr("dominant-baseline", "middle")
  .attr("fill", "white")
  .style("font-weight", 500);
```


SWINBURNE UNIVERSITY OF TECHNOLOGY

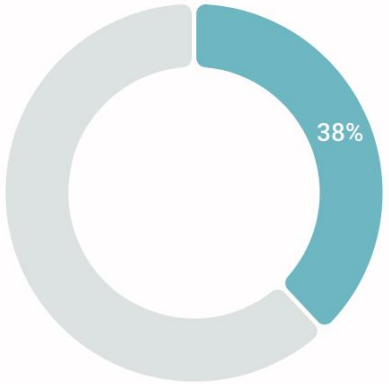
Final chart...

Melbourne City 2024 Weather

Weekly average temperature



Days with precipitation



Data source: www.bom.gov.au/climate/data/

SWINBURNE UNIVERSITY OF TECHNOLOGY